Exercices corrigés

Networking Technologies and Applications, séances 1 à 6

BMEVITMAC09, automne 2026. Rédigé le 2026-09-28, complété le 2026-10-17 (séances 4 à 6).

> **À retenir**
>
> - **Énoncés** : 68 questions en 20 niveaux. Niveaux 1 à 12 (Q1 à Q39) : séances 1 à 3, des conversions aux tables de routage, sérialisation, questions de cours. Niveaux 13 à 20 (Q40 à Q68) : routage (Dijkstra, vecteur de distance, count to infinity), transport (en-têtes, poignée de main, NAT, fenêtres TCP, BDP), Web et e-mail.
> - **Corrigés** : dans une partie séparée, avec les tableaux d'alignement binaire.

Statut. Ce ne sont pas des sujets officiels. Les seuls modèles d'exercices fournis par l'enseignant sont les quatre « Practical Examples » de `hta2p-ipv4-practical.pdf` (p. 11 à 14) et la conversion XML vers JSON ou YAML (`hta1p-protocols.pdf`, p. 26) ; les questions ci-dessous reprennent ces formats avec d'autres valeurs. Plusieurs utilisent des données réelles des slides : table `route -n`, en-tête capturé par Wireshark, fichier `kea.conf`. Les autres adresses sont des données d'exercice. Tous les résultats chiffrés ont été vérifiés avec le module Python `ipaddress` (script `exams/latex/calc_net.py`).

Séances 4 à 6 : exercices d'entraînement écrits pour ce document, au format des questions ouvertes du midterm (6 questions, 12/30 pour réussir). Ils reprennent les déroulés des slides (Dijkstra, vecteur de distance, count to infinity, fenêtres TCP, fast retransmit), les données réelles de la démo de routage (`demos/routing`), de la slide NAT et d'une sortie `netstat` du TD 2019 (`HTA_GYAK_ALL_2019.pdf`, complément en hongrois) ; graphes, en-têtes en hexadécimal, ports et débits sont des données d'exercice, signalées comme telles. Ce ne sont pas des annales. Calculs vérifiés par `exams/latex/calc_net_s46.py`, qui génère aussi les tableaux des corrigés. Les explications prises hors des slides sont marquées « rappel général ».

Méthode. Sur papier, sans calculatrice ni corrigé, en écrivant au moins l'octet intéressant en binaire. Le cours est dans « NET, cours et méthodes » ; les

numéros de concept cités renvoient à ce document.

Tableaux binaires. Bleu : bits du préfixe ; ocre : partie hôte ; | : frontière.

Table des matières

1 Énoncés **5**

1.1 Niveau 1. Conversions 5
1.2 Niveau 2. Classes et adresses spéciales 5
1.3 Niveau 3. Masques et CIDR 5
1.4 Niveau 4. Réseau, broadcast, plage d'hôtes 6
1.5 Niveau 5. Appartenance 6
1.6 Niveau 6. Découpage à taille fixe 6
1.7 Niveau 7. VLSM 7
1.8 Niveau 8. Agrégation 7
1.9 Niveau 9. Décision de routage 7
1.10 Niveau 10. En-tête et fragmentation 8
1.11 Niveau 11. Sérialisation 8
1.12 Niveau 12. Questions de cours 9
1.13 Niveau 13. Routage : notions 10
1.14 Niveau 14. Dijkstra 10
1.15 Niveau 15. Vecteur de distance 11
1.16 Niveau 16. Transport, ports, en-têtes 11
1.17 Niveau 17. NAT 12
1.18 Niveau 18. Fenêtres TCP 13
1.19 Niveau 19. Web et HTTP 14
1.20 Niveau 20. E-mail 14

2 Corrigés : conversions, classes, masques **16**

3 Corrigés : réseau, broadcast, appartenance **18**

4 Corrigés : sous-réseaux et VLSM **21**

5 Corrigés : agrégation et routage **23**

6 Corrigés : en-tête, fragmentation, sérialisation **25**

7 Corrigés : questions de cours **27**

8 Corrigés : routage **28**

9 Corrigés : transport et NAT **34**

10 Corrigés : fenêtres TCP **36**

11 Corrigés : Web et e-mail **41**

1. Énoncés

Niveau 1. Conversions

Q1. Convertir en binaire sur 8 bits : 45, 99 (par divisions successives), 250, 131.

Q2. Convertir en décimal : `10110110`, `01111111`, `11101010`, `00010101`.

Q3. Convertir 0x3F, 0xA7 et 0xFE en décimal et en binaire, puis 172 et 60 en hexadécimal.

Q4. Écrire en binaire (32 bits) et en hexadécimal les adresses 10.1.33.2 et 192.168.100.254 (deux adresses vues dans les captures de `hta3p`).

Q5. L'adresse 0xAC1F0A01 est écrite en hexadécimal. La donner en notation pointée. Est-elle privée ?

Niveau 2. Classes et adresses spéciales

Q6. Donner la classe historique et le type (privée, publique, loopback, multicast, réservée) de : 10.200.3.4 ; 172.32.1.1 ; 172.31.255.254 ; 192.168.0.1 ; 127.0.0.1 ; 224.0.0.5 ; 191.255.1.1 ; 223.1.1.1 ; 126.0.0.1 ; 240.0.0.1.

Q7. Justifier par le calcul les deux nombres de la slide pour la classe B : 16 384 réseaux et 65 534 hôtes par réseau.

Niveau 3. Masques et CIDR

Q8. Donner le masque de /13, /20, /27, /30, /9.

Q9. Donner la longueur de préfixe de 255.255.240.0, 255.128.0.0, 255.255.255.248, 255.254.0.0. Lesquels de ces masques sont invalides, et pourquoi : 255.255.0.255 ; 255.0.255.0 ; 255.255.255.160 ; 255.255.192.0 ?

Q10. Nombre d'hôtes d'un /19, d'un /23, d'un /28. Plus petit préfixe pour 300, 1000, 60, 62 et 63 hôtes.

Niveau 4. Réseau, broadcast, plage d'hôtes

Pour chaque adresse : masque, adresse réseau, broadcast, première et dernière adresse d'hôte, nombre d'hôtes.

Q11. 192.168.1.200/27

Q12. 172.16.45.14/20

Q13. 10.10.10.10/9

Q14. 131.7.255.254/23. Cette adresse peut-elle être attribuée à un hôte ?

Q15. 193.225.14.63/28. Cette adresse peut-elle être attribuée à un hôte ?

Q16. 100.64.17.130/26. Peut-on attribuer 100.64.17.191 à une autre machine du même réseau ?

Niveau 5. Appartenance

Q17. Lesquelles de ces adresses sont dans le même sous-réseau que 10.20.30.40/21 : (a) 10.20.31.200 ; (b) 10.20.24.1 ; (c) 10.20.39.1 ; (d) 10.21.30.40 ; (e) 10.20.32.0 ?

Q18. Deux machines ont les adresses 192.168.5.130 et 192.168.5.100, masque /25. Peuvent-elles échanger directement, sans routeur ? Et avec un masque /24 ?

Niveau 6. Découpage à taille fixe

Q19. Découper 192.168.30.0/24 en 4 sous-réseaux de même taille. Pour chacun : réseau, plage d'hôtes, broadcast.

Q20. Découper 10.0.0.0/16 en sous-réseaux d'au moins 1000 hôtes chacun, le plus nombreux possible. Préfixe, nombre de sous-réseaux, 5e sous-réseau (plage complète) et dernier sous-réseau.

Q21. Une entreprise veut 12 sous-réseaux égaux dans 172.18.0.0/16, les plus grands possible. Préfixe, nombre d'hôtes par sous-réseau, sous-réseau numéro 10 en comptant à partir de 0.

Niveau 7. VLSM

Q22. Plan d'adressage dans 172.16.8.0/22 pour : LAN A 400 hôtes, LAN B 120, LAN C 60, LAN D 30, deux liens point à point entre routeurs. Donner réseau, masque, plage, broadcast, et ce qui reste libre.

Q23. Plan d'adressage dans 192.168.100.0/24 pour : Production 60 hôtes, Bureaux 60, Wi-Fi 28, Serveurs 12, Administration 5.

Q24. Peut-on loger dans 200.10.10.0/24 trois réseaux de 120, 70 et 50 hôtes ? Justifier.

Niveau 8. Agrégation

Q25. Agréger 172.16.12.0/24, 172.16.13.0/24, 172.16.14.0/24 et 172.16.15.0/24, tous vers la même interface.

Q26. Table de routage : (1) 200.20.0.0/17 eth1 ; (2) 200.20.128.0/18 eth1 ; (3) 200.20.192.0/18 eth1 ; (4) 200.21.0.0/16 eth2. Compresser la table au maximum. Que deviendrait-elle si (4) sortait aussi par eth1 ?

Q27. (A) 192.168.2.0/24 et 192.168.3.0/24 vers la même interface : peut-on les fusionner ? (B) Même question pour 192.168.3.0/24 et 192.168.4.0/24. Quel est, dans le cas B, le plus petit préfixe qui couvre les deux, et pourquoi n'est-ce pas une bonne agrégation ?

Niveau 9. Décision de routage

Q28. Table : 0.0.0.0/0 vers R0 ; 152.66.0.0/16 eth1 ; 152.66.128.0/17 eth2 ; 152.66.160.0/19 eth3 ; 152.66.163.0/24 eth4. Interface de sortie pour : 152.66.163.9 ; 152.66.170.1 ; 152.66.200.1 ; 152.66.5.5 ; 152.67.0.1.

Q29. Voici la table `route -n` réelle de `hta3p-ipv4-tools.pdf`, p. 12 (colonnes Metric, Ref, Use omises) :

```
Destination     Gateway         Genmask          Flags Iface
0.0.0.0         192.168.100.254 0.0.0.0          UG    enp8s0f0
1.1.1.1         192.168.100.254 255.255.255.255  UGH   enp8s0f0
8.8.8.8         192.168.100.254 255.255.255.255  UGH   enp8s0f0
10.1.11.0       0.0.0.0         255.255.255.0    U     enp6s0
```

```
10.1.33.0        0.0.0.0         255.255.255.0    U     enp3s0
172.17.0.0       0.0.0.0         255.255.0.0      U     docker0
192.168.100.0    0.0.0.0         255.255.255.0    U     enp8s0f0
192.168.100.254  0.0.0.0         255.255.255.255  UH    enp8s0f0
```

Pour chaque destination, dire l'interface et si le paquet passe par une passerelle : 8.8.8.8 ; 10.1.33.2 ; 172.17.5.5 ; 9.9.9.9 ; 192.168.100.77 ; 10.1.12.1.

Niveau 10. En-tête et fragmentation

Q30. Voici les 20 premiers octets IP de la capture Wireshark de `hta3p`, p. 22 :

```
45 00 00 3c 6f 89 00 00 02 11 30 8d c0 a8 64 0f ac 43 47 a0
```

Donner version, longueur d'en-tête, longueur totale, Identification, drapeaux et offset, TTL, protocole, adresses source et destination. Question bonus (méthode hors slides) : vérifier la somme de contrôle.

Q31. Un paquet IPv4 de 3000 octets (en-tête de 20 octets, sans options) doit traverser un lien de MTU 1000. Donner pour chaque fragment : octets de données, Total Length, Fragment Offset, MF. Que se passe-t-il si DF vaut 1 ?

Niveau 11. Sérialisation

Q32. Convertir ce document XML (données d'exercice) en JSON puis en YAML.

```
<host>
  <name>nb</name>
  <interface>ethBA</interface>
  <lease>
    <address>10.0.1.10</address>
    <prefix>24</prefix>
    <router>10.0.1.1</router>
    <valid>20</valid>
  </lease>
  <dns>
    <server>10.0.1.1</server>
    <server>10.0.1.2</server>
  </dns>
</host>
```

Q33. Même exercice avec la table de l'exemple 3 des slides :

```
<routingTable>
  <route><dest>170.80.0.0/13</dest><iface>eth7</iface></route>
  <route><dest>170.88.0.0/13</dest><iface>eth2</iface></route>
  <route><dest>170.84.0.0/14</dest><iface>eth11</iface></route>
</routingTable>
```

Niveau 12. Questions de cours

Q34. ICMP. (a) Numéro de protocole dans l'en-tête IP. (b) Types de l'Echo Request et de l'Echo Reply. (c) Quel message les routeurs intermédiaires renvoient-ils à `traceroute`, et pourquoi ? (d) Quel message reçoit la source si DF vaut 1 et que la MTU est trop petite ? (e) Deux problèmes de sécurité d'ICMP.

Q35. DHCP. (a) Nommer les quatre messages de l'échange de base, dans l'ordre, et dire lesquels sont diffusés en broadcast. (b) Transport et ports. (c) Pourquoi le Request est-il en broadcast ? (d) Que signifie « état mou » ? (e) Dans la démo Kea, combien d'adresses le pool peut-il distribuer, et quelle passerelle est annoncée ?

Q36. DNS. (a) Types d'enregistrement pour une adresse IPv4, IPv6, un serveur de courrier. (b) Transports utilisés. (c) Comment une requête atteint-elle le serveur autoritaire ? (d) Trois outils de requête.

Q37. Outils. (a) Commande `ip` pour ajouter 10.0.20.30/24 à eth0. (b) Lister les routes en numérique. (c) Lister les connexions ouvertes avec les processus. (d) Capturer le trafic en console. (e) Lire le bilan de ping de la slide p. 18 : `24 packets transmitted, 24 received, 0% packet loss` et `rtt min/avg/max/mdev = 1.062/1.282/1.443/0.097 ms`.

Q38. Modèles en couches. (a) Nom des PDU aux couches transport, réseau, liaison et physique d'OSI. (b) À quelles couches OSI correspond la couche Application de TCP/IP ? (c) Dans quel ordre les en-têtes apparaissent-ils sur le fil pour une requête HTTP sur Ethernet ?

Q39. QoS. Un client annonce un débit moyen de 4 Mbit/s, un débit maximal de 10 Mbit/s et des rafales de 2 s. Dimensionner un token bucket selon la règle de la slide. Quelle différence de comportement avec un leaky bucket de mêmes paramètres ?

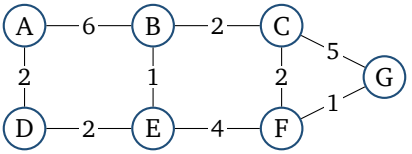
Niveau 13. Routage : notions

Q40. (a) Différence entre relayage (forwarding) et routage (routing). (b) Rôle de la FIB et de la RIB ; laquelle est optimisée pour la vitesse ? (c) Routage statique ou dynamique : définir. (d) Redondance 1:1 et 1+1 : principe, et condition pour qu'elles protègent vraiment.

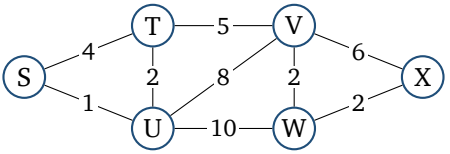
Q41. Comparer distance vector et link state : algorithme, ce qui est envoyé, à qui, vitesse de convergence, deux protocoles de chaque famille. Lequel réagit le plus vite à une panne de lien ?

Niveau 14. Dijkstra

Q42. Graphe de la slide `hta4p-routing.pdf`, p. 8 (rappelé ci-dessous). La slide déroule Dijkstra depuis C. Le dérouler depuis **A** : tableau étape par étape (distance et prédécesseur), puis table de routage de A (destination, distance, premier saut).



Q43. Graphe d'exercice ci-dessous (poids choisis pour l'entraînement). Dérouler Dijkstra depuis S. Donner les distances, les chemins et la FIB de S. Le lien direct S-T est-il utilisé ?



Q44. Démo de `hta4p`, p. 16 : quatre nœuds en carré, na (ethAB 10.0.1.1, ethAD 10.0.4.1), nb (10.0.1.2, 10.0.2.2), nc (10.0.2.3, 10.0.3.3), nd (10.0.3.4, 10.0.4.4), tous en /24. Le fichier `bird_ospf.conf` de la démo met `cost 5` sur chaque interface. (a) Écrire la table de na après convergence d'OSPF : réseau, passerelle, interface, coût. (b) Même chose pour 10.0.2.0/24 si le lien na-nb tombe.

Niveau 15. Vecteur de distance

Q45. Chaîne A-B-C-D-E-F de `hta4p`, p. 5 (RIP, métrique en sauts), tables vides au départ. (a) Table de C (destination, distance, saut suivant) après chacune des 5 premières diffusions. (b) Que contient l'annonce de C après la 2e diffusion ? Qu'est-ce qui n'y figure pas ? (c) Après combien de diffusions A connaît-il F, et pourquoi ce nombre ?

Q46. Réseau d'exercice : liens A-B coût 1, A-C 5, B-C 2, B-D 7, C-D 1. Au départ chaque nœud ne connaît que ses voisins directs. À chaque échange, tous envoient simultanément leur vecteur de l'échange précédent. (a) Donner les tables des quatre nœuds après chaque échange jusqu'à stabilité. (b) Combien d'échanges faut-il ? (c) Expliquer l'évolution de la route de A vers D.

Q47. Même chaîne qu'en Q45, convergée : C joint A en 2 sauts via B, D en 3 via C. B tombe ; C le détecte. (a) Dérouler les annonces entre C et D comme sur la slide p. 6, jusqu'à ce que les deux aient A à l'infini de RIP. Combien d'annonces ? (b) Pourquoi la boucle finit-elle avec RIP ? (c) Que se passe-t-il avec split horizon ? avec poisoned reverse ?

Q48. (a) Classer RIP, OSPF, IS-IS, BGP : IGP ou EGP, famille (DV, LS, path vector), métrique. (b) Pourquoi « 0.0.0.1 » comme identifiant d'aire OSPF n'est-il pas une adresse IP ? Rôle de l'aire 0.0.0.0 ? (c) À quoi sert un Designated Router ? (d) AS1 a un peering avec AS2, et AS2 un peering avec AS3 : AS1 peut-il joindre AS3 par AS2 ? (e) Pourquoi dit-on que BGP fait du routage par politique ?

Niveau 16. Transport, ports, en-têtes

Q49. Sortie `netstat` relevée dans le TD 2019 (`HTA_GYAK_ALL_2019.pdf`, p. 32) :

```
TCP    127.0.0.1:1906         localhost:1907         ESTABLISHED
TCP    192.168.1.147:53699    13.77.87.52:https      ESTABLISHED
TCP    192.168.1.147:53703    91.190.216.57:12350    ESTABLISHED
```

(a) Écrire le 5-uplet de la 2e ligne avec des numéros. (b) Quels ports sont éphémères, lequel est bien connu ? (c) Que montre la 1re ligne ? (d) La ma-

chine 192.168.1.147 est-elle joignable telle quelle depuis Internet ? Que fait sa passerelle ?

Q50. Voici un en-tête UDP en hexadécimal (données d'exercice) : `e1 9a 00 35 00 26 00 00`. Donner ports, longueur, taille des données, checksum. Probable service ? Le checksum est-il acceptable en IPv4, en IPv6 ?

Q51. Voici les 20 premiers octets d'un en-tête TCP (données d'exercice) :

```
01 bb c7 38 b2 d0 5e 01 00 01 e2 41 80 12 ff ff 3c 2a 00 00
```

(a) Ports source et destination. Qui est le serveur ? (b) Numéros de séquence et d'acquittement en décimal. (c) Longueur de l'en-tête et des options. (d) Drapeaux : quel segment de la poignée de main est-ce ? (e) Numéro de séquence initial du client, et numéros du segment suivant du client.

Q52. Un client ouvre une connexion avec Sc = 1000, le serveur répond avec Ss = 5000. (a) Écrire les trois segments de la poignée de main (drapeaux, Sequence, Acknowledgement). (b) Le client envoie ensuite trois segments de 100 octets : leurs numéros de séquence et l'acquittement attendu après chacun (rappel général : TCP compte en octets). (c) Écrire la fermeture lancée par le client, telle que la slide la dessine. (d) Pourquoi les numéros initiaux sont-ils aléatoires, et en quoi consiste un SYN flood ? Comment les SYN cookies le contrent-ils ?

Q53. Compléter un tableau TCP, UDP, DCCP, SCTP : numéro de protocole IP, avec ou sans connexion, datagramme ou flux, fiabilité, contrôle de congestion. Pourquoi QUIC et μTP sont-ils construits sur UDP plutôt que comme nouveaux protocoles IP ?

Niveau 17. NAT

Q54. Réseau de la slide `hta5-14.pdf`, p. 18 : passerelle 192.168.1.1 (interne) et 94.44.112.4 (externe), serveur 152.66.244.17. Trois connexions s'ouvrent (ports choisis pour l'exercice) : 192.168.1.31 port 50000 vers le port 443 ; 192.168.1.32 port 50000 vers le port 443 ; 192.168.1.31 port 50001 vers le port 80. La passerelle attribue les ports 40001, 40002, 40003 dans cet ordre. (a) Table de traduction. (b) Adresses et ports des paquets de la 2e connexion, à l'aller et au retour, de chaque côté. (c) Pourquoi la passerelle ne peut-elle pas garder le port 50000 pour les deux premières ? (d) Le

serveur peut-il ouvrir une connexion vers 192.168.1.31 ? Que faudrait-il ? (e) Deux limites du NAT citées par la slide.

Niveau 18. Fenêtres TCP

Conventions des slides `hta5p`, p. 8 et 9 : une ligne par aller-retour, `cwnd` en segments ; slow start ×2 (plafonné à `ssthresh`), congestion avoidance +1 avec `ssthresh` = `cwnd`.

Q55. Le récepteur annonce `adv_wnd` = 12 segments, constant. Départ `cwnd` = 1, `ssthresh` = `adv_wnd`. Sans perte, donner `cwnd` et la fenêtre effective sur 7 rondes. Qu'est-ce qui limite l'émission à partir de la ronde 5 ?

Q56. TCP Tahoe, `ssthresh` initial = 8, `cwnd` = 1. Pendant la ronde 12, un segment est perdu et détecté par **timeout**. (a) Tableau ronde, `cwnd`, `ssthresh`, phase, des rondes 1 à 18. (b) Nombre total de segments émis sur les 18 rondes.

Q57. Même scénario avec TCP **Reno**, mais la perte de la ronde 12 est détectée par **3 ACK dupliqués**. (a) Même tableau. (b) Total sur 18 rondes et gain par rapport à Q56. (c) Relier ce gain au schéma de la slide p. 15. (d) Que ferait Reno si la perte était détectée par timeout ?

Q58. Figure de `hta5p`, p. 13 : l'émetteur envoie les segments 20 à 26, le 21 est perdu ; le récepteur acquitte en donnant le prochain segment attendu. (a) Liste des ACK. (b) À quel moment l'émetteur retransmet-il 21 ? (c) Quel ACK suit la réception de 21, et pourquoi ce saut ? (d) Pourquoi attendre 3 doublons plutôt qu'un ? Que fait New Reno ?

Q59. Produit bande passante délai. (a) Lien de 100 Mbit/s, RTT 80 ms : BDP en octets et en segments de 1460 octets ; débit maximal avec la fenêtre maximale sans option ; plus petit décalage window scale suffisant. (b) Même chose pour 10 Gbit/s et RTT 100 ms ; la limite de 1 Go suffit-elle ? (c) Le TD 2019 (p. 40) compare une fenêtre de 8 ko et de 64 ko : débit maximal de chacune avec un RTT de 80 ms.

Q60. Complément TD 2019, p. 55 : ligne de 10 Mbit/s, taux d'erreur binaire 10^{-6}, paquets de 1500 octets. (a) Probabilité de perdre un paquet, et « un paquet sur combien ». (b) Pourquoi TCP Reno exploite-t-il mal un tel lien ? (c) Trois remèdes cités par `hta5p`, p. 17.

Q61. Questions de cours. (a) Contrôle de flux et contrôle de congestion : qui protège-t-on, quel champ ou quelle variable ? (b) Que fait l'émetteur quand `adv_wnd` $= 0$? (c) Silly Window Syndrome : cause et deux remèdes. (d) Goodput, genou, falaise. (e) Deux façons de détecter une perte.

Niveau 19. Web et HTTP

Q62. (a) Classer 200, 301, 403, 404, 500, 502, 504. (b) GET, POST, HEAD : rôle. (c) HTTP est sans état : comment une boutique garde-t-elle un panier ? Qu'est-ce qu'un cookie tiers, et pourquoi le contenu statique est-il servi depuis un domaine sans cookie ?

Q63. (a) Nouveauté principale de HTTP/1.1, HTTP/2, HTTP/3. (b) Page de 5 objets sur un même serveur, RTT 50 ms, objets demandés l'un après l'autre, temps d'émission négligé : temps total avec HTTP/1.0 sans réutilisation, puis HTTP/1.1 (rappel général, modèle de Kurose et Ross). (c) Combien d'allers-retours avant la première requête avec TCP + TLS 1.3, puis avec QUIC ? Temps pour un RTT de 50 ms (rappel général).

Q64. HTTPS. (a) Ports HTTP et HTTPS. (b) Pourquoi Diffie-Hellman seul ne protège-t-il pas contre l'homme du milieu ? (c) Les deux conditions vérifiées par le navigateur sur le certificat d'un site. (d) Certificat auto-signé : à quoi sert-il, à quoi ne sert-il pas ? (e) Web of trust et PKI. (f) À quelle couche la slide place-t-elle TLS, et dans quel ordre se font les poignées de main ?

Niveau 20. E-mail

Q65. Alice écrit à Bob depuis Thunderbird ; Bob lit son courrier en IMAP. Adresses (domaines d'exercice) : `alice@a.example` et `bob@b.example`. Décrire la chaîne : agents traversés (MUA, MSA, MTA, MDA), protocole et port à chaque étape, rôle du DNS.

Q66. (a) Ordre des commandes SMTP de la slide. (b) POP3 et IMAP : ports en clair et avec TLS, deux différences de fonctionnement. (c) Sur quoi repose en général un webmail ?

Q67. (a) Pourquoi MIME est-il nécessaire ? Deux encodages. (b) Taille approximative d'une pièce jointe de 3 Mo encodée en base64 (rappel général : 3 octets donnent 4 caractères). (c) mbox et maildir : différence.

Q68. (a) PLAIN et CRAM : principe, et contre quoi CRAM protège. (b) TOTP : ce qui est partagé, durée de validité, condition technique. (c) Associer DKIM, SPF, DMARC à leur rôle. (d) Sur quoi repose la confiance dans PGP ? (e) Pourquoi la slide juge-t-elle impossible une protection totale contre le phishing ?

2. Corrigés : conversions, classes, masques

Q1. 45 : 32+8+4+1 : `00101101`. 250 : 128+64+32+16+8+2 : `11111010`. 131 : 128 + 2 + 1 : `10000011`.

99 par divisions : 99 = 2×49+1 ; 49 = 2×24+1 ; 24 = 2×12+0 ; 12 = 2×6+0 ; 6 = 2 × 3 + 0 ; 3 = 2 × 1 + 1 ; 1 = 2 × 0 + 1. Restes de bas en haut : `1100011`, soit `01100011` sur 8 bits (64 + 32 + 2 + 1 = 99).

Q2. `10110110` = 128 + 32 + 16 + 4 + 2 = 182 ; `01111111` = 127 ; `11101010` = 128 + 64 + 32 + 8 + 2 = 234 ; `00010101` = 16 + 4 + 1 = 21.

Q3. 0x3F = 3 × 16 + 15 = 63, `0011 1111`. 0xA7 = 10 × 16 + 7 = 167, `1010 0111`. 0xFE = 15 × 16 + 14 = 254, `1111 1110`. 172 = 10 × 16 + 12 : 0xAC. 60 = 3 × 16 + 12 : 0x3C.

Q4.

Adresse	Binaire	Hex
10.1.33.2	`00001010.00000001.00100001.00000010`	0x0A012102
192.168.100.254	`11000000.10101000.01100100.11111110`	0xC0A864FE

33 = 32 + 1 ; 100 = 64 + 32 + 4 = 0x64 ; 254 = 255 − 1 = 0xFE.

Q5. AC, 1F, 0A, 01 : 10 × 16 + 12 = 172, 16 + 15 = 31, 10, 1. Adresse **172.31.10.1**. Privée : 172.16.0.0/12 couvre 172.16 à 172.31, et 31 = `00011111` commence bien par `0001`.

Q6.

Adresse	1er octet	Classe, type
10.200.3.4	00001010	A, privée (10.0.0.0/8)
172.32.1.1	10101100	B, **publique** (32 est hors de 16 à 31)
172.31.255.254	10101100	B, privée
192.168.0.1	11000000	C, privée
127.0.0.1	01111111	loopback (réseau 127 réservé)
224.0.0.5	11100000	D, multicast
191.255.1.1	10111111	B, publique
223.1.1.1	11011111	C, publique
126.0.0.1	01111110	A, publique
240.0.0.1	11110000	E, réservée

Q7. Classe B : préfixe de 16 bits dont les 2 premiers sont fixés à 10, il reste 14 bits libres : $2^{14} = 16\,384$ réseaux. Partie hôte de 16 bits : $2^{16} = 65\,536$ combinaisons, moins l'adresse de réseau (tout à 0) et le broadcast (tout à 1) : 65 534.

Q8. /13 = 8 + 5 : 255.248.0.0. /20 = 16 + 4 : 255.255.240.0. /27 = 24 + 3 : 255.255.255.224. /30 = 24 + 6 : 255.255.255.252. /9 = 8 + 1 : 255.128.0.0.

Q9. 255.255.240.0 : 16 + 4 = /20. 255.128.0.0 : 8 + 1 = /9. 255.255.255.248 : 24 + 5 = /29. 255.254.0.0 : 8 + 7 = /15.

Invalides : 255.255.0.255 (00000000 suivi de 11111111) ; 255.0.255.0 (même défaut) ; 255.255.255.160 (160 = 10100000, un 0 entre deux 1). 255.255.192.0 est valide : /18.

Q10. /19 : $2^{13} - 2 = 8190$. /23 : $2^9 - 2 = 510$. /28 : $2^4 - 2 = 14$.

300 hôtes : $2^9 - 2 = 510 \geq 300$ et $2^8 - 2 = 254 < 300$: /23. 1000 : $2^{10} - 2 = 1022$: /22. 60 : $2^6 - 2 = 62$: /26. 62 : /26 aussi, tout juste. 63 : $62 < 63$, il faut $2^7 - 2 = 126$: /25.

3. Corrigés : réseau, broadcast, appartenance

Q11.

Adresse	192.168.1.200	11000000.10101000.00000001.110|01000
Masque /27	255.255.255.224	11111111.11111111.11111111.111|00000
Réseau (ET)	192.168.1.192	11000000.10101000.00000001.110|00000
Broadcast	192.168.1.223	11000000.10101000.00000001.110|11111

Masque 255.255.255.224, $m = 32$; $192 \leq 200 < 224$. Réseau **192.168.1.192**, broadcast **192.168.1.223**, hôtes 192.168.1.193 à 192.168.1.222, soit 30.

Q12.

Adresse	172.16.45.14	10101100.00010000.0010|1101.00001110
Masque /20	255.255.240.0	11111111.11111111.1111|0000.00000000
Réseau (ET)	172.16.32.0	10101100.00010000.0010|0000.00000000
Broadcast	172.16.47.255	10101100.00010000.0010|1111.11111111

Masque 255.255.240.0, octet intéressant le 3e, $m = 16$; $32 \leq 45 < 48$; en binaire $45 = 00101101$, ET $11110000 = 00100000 = 32$. Réseau **172.16.32.0**, broadcast **172.16.47.255**, hôtes 172.16.32.1 à 172.16.47.254, soit $2^{12} - 2 = 4094$.

Q13.

Adresse	10.10.10.10	00001010.0|0001010.00001010.00001010
Masque /9	255.128.0.0	11111111.1|0000000.00000000.00000000
Réseau (ET)	10.0.0.0	00001010.0|0000000.00000000.00000000
Broadcast	10.127.255.255	00001010.0|1111111.11111111.11111111

Masque 255.128.0.0, 2e octet, $m = 128$; $0 \leq 10 < 128$. Réseau **10.0.0.0**, broadcast **10.127.255.255**, hôtes 10.0.0.1 à 10.127.255.254, soit $2^{23} - 2 = 8\,388\,606$.

Q14.

Adresse	131.7.255.254	10000011.00000111.1111111|1.11111110
Masque /23	255.255.254.0	11111111.11111111.1111111|0.00000000
Réseau (ET)	131.7.254.0	10000011.00000111.1111111|0.00000000
Broadcast	131.7.255.255	10000011.00000111.1111111|1.11111111

Masque 255.255.254.0, 3e octet, $m = 2$; $254 \leq 255 < 256$. Réseau **131.7.254.0**, broadcast **131.7.255.255**, hôtes 131.7.254.1 à 131.7.255.254, soit 510. L'adresse 131.7.255.254 est la **dernière adresse d'hôte** : attribuable. Un 4e octet à 255 ou 0 n'est pas forcément interdit dans un réseau plus grand qu'un /24 : 131.7.254.255 est ici une adresse d'hôte ordinaire.

Q15.

Adresse	193.225.14.63	11000001.11100001.00001110.0011|1111
Masque /28	255.255.255.240	11111111.11111111.11111111.1111|0000
Réseau (ET)	193.225.14.48	11000001.11100001.00001110.0011|0000
Broadcast	193.225.14.63	11000001.11100001.00001110.0011|1111

Masque 255.255.255.240, $m = 16$; $48 \leq 63 < 64$. Réseau 193.225.14.48, broadcast **193.225.14.63** : c'est l'adresse donnée. **Non attribuable**. Hôtes .49 à .62, soit 14.

Q16.

Adresse	100.64.17.130	01100100.01000000.00010001.10|000010
Masque /26	255.255.255.192	11111111.11111111.11111111.11|000000
Réseau (ET)	100.64.17.128	01100100.01000000.00010001.10|000000
Broadcast	100.64.17.191	01100100.01000000.00010001.10|111111

Masque 255.255.255.192, $m = 64$; $128 \leq 130 < 192$. Réseau 100.64.17.128, broadcast 100.64.17.191, hôtes .129 à .190, soit 62. 100.64.17.191 est le broadcast : **non**.

Q17.

Réf.	10.20.30.40	00001010.00010100.00011|110.00101000
(a)	10.20.31.200	00001010.00010100.00011|111.11001000
(b)	10.20.24.1	00001010.00010100.00011|000.00000001
(c)	10.20.39.1	00001010.00010100.00100|111.00000001
(d)	10.21.30.40	00001010.00010101.00011|110.00101000
(e)	10.20.32.0	00001010.00010100.00100|000.00000000

Réseau de référence : $m = 8$ au 3e octet, $24 \leq 30 < 32$: 10.20.24.0/21, plage 10.20.24.0 à 10.20.31.255 (5 premiers bits du 3e octet : 00011). (a) 31 = 00011111 : **oui**. (b) 24 = 00011000 : **oui**. (c) 39 = 00100111 : **non**, réseau 10.20.32.0/21. (d) 2e octet 21 : **non**. (e) 32 = 00100000 : **non**, c'est même l'adresse du réseau voisin 10.20.32.0/21.

Q18.

Hôte 1	192.168.5.130	11000000.10101000.00000101.1|0000010
Hôte 2	192.168.5.100	11000000.10101000.00000101.0|1100100

En /25, le 25e bit diffère : 1 pour .130, 0 pour .100. Les réseaux sont 192.168.5.128/25 et 192.168.5.0/25. **Pas d'échange direct**, il faut passer par un routeur. En /24, les 24 premiers bits sont identiques : même réseau 192.168.5.0/24, échange direct.

4. Corrigés : sous-réseaux et VLSM

Q19. 4 sous-réseaux : $s = 2$ bits, préfixe /26, masque 255.255.255.192, 62 hôtes chacun.

Bits empruntés	Réseau	Hôtes	Broadcast
00	192.168.30.0/26	.1 à .62	.63
01	192.168.30.64/26	.65 à .126	.127
10	192.168.30.128/26	.129 à .190	.191
11	192.168.30.192/26	.193 à .254	.255

Q20. 1000 hôtes : $2^{10} - 2 = 1022$, donc $h = 10$, préfixe **/22** (255.255.252.0). Bits empruntés : $22 - 16 = 6$, soit $2^6 = \mathbf{64}$ sous-réseaux, pas de 4 au 3e octet. 5e sous-réseau ($k = 4$) : $4 \times 4 = 16$: **10.0.16.0/22**, hôtes 10.0.16.1 à 10.0.19.254, broadcast 10.0.19.255. Dernier ($k = 63$) : $63 \times 4 = 252$: **10.0.252.0/22**, broadcast 10.0.255.255.

Q21. $2^3 = 8 < 12 \leq 16 = 2^4$: $s = 4$, préfixe **/20** (255.255.240.0), 16 sous-réseaux (4 de réserve), $2^{12} - 2 = \mathbf{4094}$ hôtes chacun, pas de 16 au 3e octet. $k = 10$: $10 \times 16 = 160$: **172.18.160.0/20**, hôtes 172.18.160.1 à 172.18.175.254, broadcast 172.18.175.255.

Q22. Tailles : 400 → 512 (/23), 120 → 128 (/25), 60 → 64 (/26), 30 → 32 (/27), liens → 4 (/30). Total $512 + 128 + 64 + 32 + 4 + 4 = 744 \leq 1024$.

Réseau	Préfixe	Masque	Plage d'hôtes	Broadcast
LAN A	172.16.8.0/23	255.255.254.0	8.1 à 9.254	9.255
LAN B	172.16.10.0/25	255.255.255.128	10.1 à 10.126	10.127
LAN C	172.16.10.128/26	255.255.255.192	10.129 à 10.190	10.191
LAN D	172.16.10.192/27	255.255.255.224	10.193 à 10.222	10.223
Lien 1	172.16.10.224/30	255.255.255.252	10.225, 10.226	10.227
Lien 2	172.16.10.228/30	255.255.255.252	10.229, 10.230	10.231

(Plages abrégées : « 8.1 » signifie 172.16.8.1.) Libre : de 172.16.10.232 à 172.16.11.255.

Q23. Tailles : 60 → 64 (/26, 62 hôtes) deux fois, 28 → 32 (/27), 12 → 16 (/28), 5 → 8 (/29, 6 hôtes). Total $64 + 64 + 32 + 16 + 8 = 184 \leq 256$.

Réseau	Préfixe	Hôtes	Broadcast
Production	192.168.100.0/26	.1 à .62	.63
Bureaux	192.168.100.64/26	.65 à .126	.127
Wi-Fi	192.168.100.128/27	.129 à .158	.159
Serveurs	192.168.100.160/28	.161 à .174	.175
Administration	192.168.100.176/29	.177 à .182	.183

Libre : 192.168.100.184 à 192.168.100.255.

Q24. 120 demande un /25 (126 hôtes, 128 adresses). 70 ne tient pas dans un /26 (62 hôtes) : il faut aussi un /25. Deux /25 occupent déjà les 256 adresses du /24 ; il ne reste rien pour les 50 hôtes (un /26 de 64 adresses). Total nécessaire $128 + 128 + 64 = 320 > 256$: **impossible**.

5. Corrigés : agrégation et routage

Q25.

/24	172.16.12.0	10101100.00010000.000011|00.00000000
/24	172.16.13.0	10101100.00010000.000011|01.00000000
/24	172.16.14.0	10101100.00010000.000011|10.00000000
/24	172.16.15.0	10101100.00010000.000011|11.00000000
Résumé /22	172.16.12.0	10101100.00010000.000011|00.00000000

12 = 00001100, 15 = 00001111 : les 6 premiers bits du 3e octet sont communs, les 2 derniers prennent les 4 valeurs. Résumé exact : **172.16.12.0/22**. (12 est un multiple de 4 : le bloc est aligné.)

Q26.

(1) /17	200.20.0.0	11001000.00010100.0|0000000.00000000
(2) /18	200.20.128.0	11001000.00010100.10|000000.00000000
(3) /18	200.20.192.0	11001000.00010100.11|000000.00000000
(4) /16	200.21.0.0	11001000.00010101.|00000000.00000000

(2) et (3) : /18, 3e octets 10000000 et 11000000, seul le 18e bit diffère, même interface : fusion en 200.20.128.0/17. Avec (1) 200.20.0.0/17 : seul le 17e bit diffère : fusion en **200.20.0.0/16** vers eth1. (4) 200.21.0.0/16 sort par eth2 : pas de fusion. Table finale : 200.20.0.0/16 eth1 ; 200.21.0.0/16 eth2.

Si (4) sortait par eth1 : 20 = 00010100 et 21 = 00010101 ne diffèrent que par le 16e bit : une seule ligne, **200.20.0.0/15** eth1.

Q27.

A	192.168.2.0	11000000.10101000.0000001|0.00000000
B	192.168.3.0	11000000.10101000.0000001|1.00000000
C	192.168.3.0	11000000.10101000.00000|011.00000000
D	192.168.4.0	11000000.10101000.00000|100.00000000

(A) 2 = 00000010, 3 = 00000011 : seul le 24e bit diffère, fusion en **192.168.2.0/23**. (B) 3 = 00000011, 4 = 00000100 : ils diffèrent dès le 22e bit. Contigus mais non alignés : pas de /23. Le plus petit préfixe commun est **192.168.0.0/21** (192.168.0.0 à 192.168.7.255), qui couvre aussi 192.168.0, 1, 2, 5, 6 et 7 : on enverrait vers cette interface du trafic qui ne

lui est pas destiné. Il faut garder les deux lignes.

Q28.

Destination	Lignes qui correspondent	Sortie
152.66.163.9	/0, /16, /17, /19, /24	eth4 (/24)
152.66.170.1	/0, /16, /17, /19	eth3 (/19)
152.66.200.1	/0, /16, /17	eth2 (/17)
152.66.5.5	/0, /16	eth1 (/16)
152.67.0.1	/0	R0 (défaut)

Repères : /17 couvre 152.66.128.0 à 152.66.255.255 ; /19 couvre 152.66.160.0 à 152.66.191.255 ($m = 32$ au 3e octet) ; /24 couvre 152.66.163.x. 170 est entre 160 et 191 mais 200 non.

Q29.

Destination	Ligne retenue	Résultat
8.8.8.8	8.8.8.8/32, UGH	enp8s0f0, via 192.168.100.254
10.1.33.2	10.1.33.0/24, U	enp3s0, directe
172.17.5.5	172.17.0.0/16, U	docker0, directe
9.9.9.9	0.0.0.0/0, UG	enp8s0f0, via 192.168.100.254
192.168.100.77	192.168.100.0/24, U	enp8s0f0, directe
10.1.12.1	0.0.0.0/0, UG	enp8s0f0, via 192.168.100.254

Pour 8.8.8.8, la route par défaut correspond aussi, mais le /32 est plus long. 10.1.12.1 n'est ni dans 10.1.11.0/24 ni dans 10.1.33.0/24 : seule la route par défaut reste.

6. Corrigés : en-tête, fragmentation, sérialisation

Q30.

Octets	Champ	Valeur
`45`	version, IHL	4 ; $5 \times 4 = 20$ octets
`00`	DSCP, ECN	0, 0
`00 3c`	Total Length	60 octets (40 de données UDP)
`6f 89`	Identification	$6 \times 4096 + 15 \times 256 + 8 \times 16 + 9 = 28\,553$
`00 00`	Flags, Offset	DF = 0, MF = 0, offset 0
`02`	TTL	2
`11`	Protocol	17, UDP
`30 8d`	Checksum	0x308D
`c0 a8 64 0f`	Source	192.168.100.15
`ac 43 47 a0`	Destination	172.67.71.160

Ce sont les valeurs affichées par Wireshark sur la slide.

Bonus (RFC 1071, non traité dans les slides) : additionner les dix mots de 16 bits : 0x4500 + 0x003C + 0x6F89 + 0x0000 + 0x0211 + 0x308D + 0xC0A8 + 0x640F + 0xAC43 + 0x47A0 = 0x2FFFD. Replier la retenue : 0xFFFD + 0x2 = 0xFFFF. Le résultat tout à 1 signifie que l'en-tête est intact. (Sans le champ de contrôle, la somme repliée vaut 0xCF72, dont le complément à 1 est bien 0x308D.)

Q31. Données : $3000 - 20 = 2980$. Par fragment : $1000 - 20 = 980$, ramené au multiple de 8 inférieur : $\lfloor 980/8 \rfloor \times 8 = 122 \times 8 = 976$.

Fragment	Données	Total Length	Offset	MF
1	976	996	0	1
2	976	996	122	1
3	976	996	244	1
4	52	72	366	0

Contrôle : $3 \times 976 + 52 = 2980$; $366 \times 8 = 2928$ octets déjà envoyés. Tous gardent la même Identification. Si DF = 1 : le routeur jette le paquet et

peut renvoyer un ICMP Destination Unreachable, sous-type « fragmentation needed » (type 3).

Q32. Racine `host` omise, comme sur la slide ; `server` répété devient une liste ; `prefix` et `valid` sont des nombres.

```
{
  "name": "nb",
  "interface": "ethBA",
  "lease": {
    "address": "10.0.1.10",
    "prefix": 24,
    "router": "10.0.1.1",
    "valid": 20
  },
  "dns": {
    "server": [ "10.0.1.1",
                "10.0.1.2" ]
  }
}
```

```
name: nb
interface: ethBA
lease:
  address: 10.0.1.10
  prefix: 24
  router: 10.0.1.1
  valid: 20
dns:
  server:
    - 10.0.1.1
    - 10.0.1.2
```

Variante acceptable : `"dns": ["10.0.1.1", "10.0.1.2"]`, en supprimant la balise répétée comme la slide le fait pour `SlideTitle`. Annoncer le choix.

Q33. Chaque `route` est un élément composé : la liste contient des objets.

```
{
  "route": [
    { "dest": "170.80.0.0/13",
      "iface": "eth7" },
    { "dest": "170.88.0.0/13",
      "iface": "eth2" },
    { "dest": "170.84.0.0/14",
      "iface": "eth11" }
  ]
}
```

```
route:
  - dest: 170.80.0.0/13
    iface: eth7
  - dest: 170.88.0.0/13
    iface: eth2
  - dest: 170.84.0.0/14
    iface: eth11
```

En YAML, le tiret ouvre un élément de liste et les clés suivantes, alignées sous la première, appartiennent au même objet.

7. Corrigés : questions de cours

Source : Q34 : `hta3` p. 4 à 7 ; Q35 : `hta3` p. 8 à 10 et `kea.conf` ; Q36 : `hta3` p. 11 à 18 ; Q37 : `hta3p` p. 9 à 18 ; Q38 : `hta1p` p. 3 à 6 ; Q39 : `hta1p` p. 18 et 19

Q34. (a) Protocol = 1. (b) Echo Request type 8, Echo Reply type 0. (c) Time Exceeded (type 11), car le TTL, envoyé à 1, 2, 3…, tombe à 0 sur le routeur de rang correspondant ; chaque réponse révèle un saut. (d) Destination Unreachable (type 3), sous-type « fragmentation needed ». (e) Au choix : pas d'authentification (faux Redirect), ping flood, tunnel ping, cartographie.

Q35. (a) Discover (broadcast liaison et IP), Offer, Request (broadcast), Ack. (b) UDP, serveur 67, client 68. (c) Pour prévenir tous les serveurs de l'offre retenue. (d) Le bail expire s'il n'est pas renouvelé ; utiliser une adresse expirée pose problème si le serveur l'a redonnée. (e) Pool 10.0.1.10 à 10.0.1.30 : $30 - 10 + 1 = 21$ adresses ; passerelle (`routers`) 10.0.1.1.

Q36. (a) A, AAAA, MX. (b) UDP/53 d'origine, TCP/53 (messages longs, fiabilité), TLS, HTTPS. (c) Le client interroge son résolveur local (souvent appris par DHCP) ; la requête monte dans l'arbre puis redescend jusqu'au serveur autoritaire, ou s'arrête à une réponse en cache. (d) `host`, `nslookup`, `dig`.

Q37. (a) `ip addr add 10.0.20.30/24 dev eth0`. (b) `route -n` (ou `netstat -r`). (c) `netstat -pnaut`. (d) `tcpdump` (ou `tshark`). (e) 24 requêtes, 24 réponses, aucune perte ; aller-retour minimal 1,062 ms, moyen 1,282 ms, maximal 1,443 ms, écart moyen 0,097 ms.

Q38. (a) Segment, Packet, Frame, Bit. (b) Application, Présentation et une partie de Session (selon le schéma de la slide p. 6). (c) Ethernet, IP, TCP, puis HTTP : la couche basse encapsule en dernier.

Q39. $K = 4$ Mbit/s (débit moyen) ; $B = (10 - 4) \times 2 = 12$ Mbit. Le token bucket laisse passer les rafales tant qu'il reste des jetons et rejette au-delà (mesure, passe ou échoue). Le leaky bucket met en forme : la sortie ne dépasse jamais K, l'excès attend dans une file et n'est rejeté que seau plein.

8. Corrigés : routage

Source : Q40 : `hta4` p. 2 et 5; Q41 : `hta4` p. 7 à 9; Q42 à Q44 : `hta4p` p. 3 et 8 à 16, `demos/routing`; Q45 à Q47 : `hta4p` p. 4 à 6; Q48 : `hta4` p. 10 à 15. Calculs : `calc_net_s46.py`

Q40. (a) Relayer : choisir le saut suivant d'un paquet d'après ce qu'on sait de la topologie. Router : découvrir cette topologie. (b) La FIB sert directement au choix du saut suivant et est optimisée pour la vitesse de recherche ; la RIB sert à la découverte de la topologie et peut contenir des informations annexes (méthode de découverte, expiration). (c) Statique : routes saisies à la main ; dynamique : apprises par un protocole de routage. (d) 1:1 : un chemin de secours, bascule à la panne du principal ; 1+1 : chaque paquet part sur les deux chemins. Les deux ne protègent que si les chemins sont **disjoints** (sinon une seule panne coupe les deux).

Q41. DV : Bellman-Ford distribué ; on envoie à ses **voisins** la liste des destinations connues avec leur distance ; convergence lente (une distance de plus par ronde) ; RIP, IGRP. LS : Dijkstra ; on envoie à **tous les nœuds** l'état de ses liens (qui sont ses voisins) ; chacun calcule sur la topologie complète ; OSPF, IS-IS. LS détecte et reconfigure plus vite après une panne de lien.

Q42.

Étape	Retiré	B	C	D	E	F	G
0	A (0)	6,A	∞	2,A	∞	∞	∞
1	D (2)	6,A	∞	**2**	4,D	∞	∞
2	E (4)	5,E	∞		**4**	8,E	∞
3	B (5)	**5**	7,B			8,E	∞
4	C (7)		**7**			8,E	12,C
5	F (8)					**8**	9,F
6	G (9)						**9**

Destination	Distance	Premier saut	Chemin
B	5	D	A → D → E → B
C	7	D	A → D → E → B → C
D	2	D	A → D
E	4	D	A → D → E
F	8	D	A → D → E → F
G	9	D	A → D → E → F → G

À l'étape 2, E retiré à 4 améliore B (via A-D-E-B = 5 < 6) : le lien direct A-B n'est pas utilisé. À l'étape 4, C donne G à 12, que F améliore à 9 à l'étape 5. Tous les premiers sauts de A sont D : la FIB de A se résume à « tout via D ».

Q43.

Étape	Retiré	T	U	V	W	X
0	S (0)	4,S	1,S	∞	∞	∞
1	U (1)	3,U	**1**	9,U	11,U	∞
2	T (3)	**3**		8,T	11,U	∞
3	V (8)			**8**	10,V	14,V
4	W (10)				**10**	12,W
5	X (12)					**12**

Destination	Distance	Premier saut	Chemin
T	3	U	S → U → T
U	1	U	S → U
V	8	U	S → U → T → V
W	10	U	S → U → T → V → W
X	12	U	S → U → T → V → W → X

Le lien direct S-T (4) n'est **pas** utilisé : S-U-T coûte 1 + 2 = 3. FIB de S : tout passe par U.

Q44. Rappel général (RFC 2328) : le coût OSPF d'une route est la somme des coûts des interfaces de sortie traversées, y compris celle qui donne sur le réseau de destination. La passerelle est l'adresse du voisin sur le lien commun.

Réseau	Passerelle	Interface	Coût
10.0.1.0/24	directe	ethAB	5
10.0.2.0/24	10.0.1.2 (nb)	ethAB	5 + 5 = 10
10.0.3.0/24	10.0.4.4 (nd)	ethAD	5 + 5 = 10
10.0.4.0/24	directe	ethAD	5

Par l'autre côté du carré, 10.0.2.0/24 coûterait 15 (na, nd, nc), donc nb est choisi. (b) Lien na-nb coupé : 10.0.2.0/24 via 10.0.4.4, interface ethAD, coût 5 + 5 + 5 = 15 (na, nd, nc). C'est ce que montre la démo : BIRD ajoute les routes à la FIB et tout le monde se joint (`hta4p`, p. 17).

Q45. (a)

Après la diffusion	A	B	D	E	F
1		1 via B	1 via D		
2	2 via B	1 via B	1 via D	2 via D	
3	2 via B	1 via B	1 via D	2 via D	3 via D
4	2 via B	1 via B	1 via D	2 via D	3 via D
5	2 via B	1 via B	1 via D	2 via D	3 via D

(b) Après la 2e diffusion, C annonce `[[A,2],[B,1],[D,1],[E,2]]` : destinations et distances, **sans le saut suivant**, que C garde pour lui. (c) Après la 5e : chaque diffusion fait connaître les nœuds un saut plus loin, et A et F sont à 5 sauts. C'est la convergence lente du DV : autant de rondes que le diamètre du réseau.

Q46. Règle : $d_X(Y) = \min_v \big(c(X,v) + d_v(Y)\big)$ sur les voisins v, avec les vecteurs reçus à l'échange précédent.

Échange	Nœud	vers A	vers B	vers C	vers D
0	A	0	1 via B	5 via C	∞
	B	1 via A	0	2 via C	7 via D
	C	5 via A	2 via B	0	1 via D
	D	∞	7 via B	1 via C	0
1	A	0	1 via B	3 via B	6 via C
	B	1 via A	0	2 via C	3 via C
	C	3 via B	2 via B	0	1 via D
	D	6 via C	3 via C	1 via C	0
2	A	0	1 via B	3 via B	4 via B
	B	1 via A	0	2 via C	3 via C
	C	3 via B	2 via B	0	1 via D
	D	4 via C	3 via C	1 via C	0
3	A	0	1 via B	3 via B	4 via B
	B	1 via A	0	2 via C	3 via C
	C	3 via B	2 via B	0	1 via D
	D	4 via C	3 via C	1 via C	0

(b) Les tables ne changent plus entre l'échange 2 et l'échange 3 : stables après **2** échanges (le 3e le constate). (c) Au départ A ne connaît pas D (∞). Échange 1 : C annonce D à 1, B annonce D à 7 : $\min(5+1,\ 1+7) = 6$ via C. Échange 2 : B a appris D à 3 via C : $\min(1+3,\ 5+1) = 4$ via B. La meilleure route A-B-C-D ($1+2+1$) a demandé deux échanges, car elle traverse deux intermédiaires.

Q47. (a)

Message	Annonce	C vers A	D vers A
panne de B		inaccessible	3 via C
1	D annonce A à 3	4 via D	3 via C
2	C annonce A à 4	4 via D	5 via C
3	D annonce A à 5	6 via D	5 via C
4	C annonce A à 6	6 via D	7 via C
5	D annonce A à 7	8 via D	7 via C
6	C annonce A à 8	8 via D	9 via C
7	D annonce A à 9	10 via D	9 via C
8	C annonce A à 10	10 via D	11 via C
9	D annonce A à 11	12 via D	11 via C
10	C annonce A à 12	12 via D	13 via C
11	D annonce A à 13	14 via D	13 via C
12	C annonce A à 14	14 via D	15 via C
13	D annonce A à 15	16 (infini RIP)	15 via C
14	C annonce A à 16	16 (infini RIP)	16 (infini RIP)

Il faut **14** annonces. C ne sait pas qu'il est le saut suivant de D ; D, dont C est le saut suivant, accepte toute nouvelle valeur de C. (b) RIP limite la métrique à 15 sauts ; 16 vaut infini (rappel général, RFC 1058), ce qui arrête la boucle. Avec une annonce toutes les 30 s, cela prend des minutes. (c) Split horizon : D n'annonce pas A à C, puisque C est son saut suivant vers A ; C garde A inaccessible et le dit à D, qui marque A inaccessible : propagation en une annonce. Poisoned reverse : D annonce A à C avec un coût infini ; C ne choisit pas D. Même résultat.

Q48. (a) RIP : IGP, DV, sauts (15 au plus). OSPF : IGP, LS, coût (souvent bande passante relative à une référence). IS-IS : IGP, LS, pas lié à IP, réseaux d'opérateurs. BGP : EGP, path vector, politique (coût en argent, confiance). (b) L'identifiant d'aire est un nombre de 32 bits seulement *écrit* a.b.c.d. L'aire 0.0.0.0 est la dorsale qui relie toutes les autres aires ; les LSA restent dans leur aire. (c) Sur un lien multipoint (Ethernet), il représente le segment, avec un Backup DR. (d) Non : le peering est une relation par paire, **non transitive** ; il faudrait un transit (contrat). (e) Les routes mêlent découverte automatique et directives manuelles : on paie le trafic, on contourne un pays

soupçonné d’espionnage.

9. Corrigés : transport et NAT

Source : Q49 : hta5 p. 4, TD 2019 p. 32, hta6 p. 12 ; Q50 : hta5 p. 7 ; Q51, Q52 : hta5 p. 9 à 16 ; Q53 : hta5 p. 20 à 23 ; Q54 : hta5 p. 18 et 19. Décodage vérifié par `calc_net_s46.py`

Q49. (a) {192.168.1.147, 13.77.87.52, TCP, 53699, 443} (https est le port 443, hta6, p. 12). (b) 53699 et 53703 sont des ports éphémères ouverts par le système côté client ; 443 est bien connu (0 à 1023) ; 12350 est vraisemblablement le port d'écoute du serveur distant, hors de la plage bien connue, ce que la slide autorise. (c) Une connexion locale entre deux processus de la même machine (127.0.0.1 est le loopback). (d) Non, 192.168.1.147 est privée (192.168.0.0/16), non routable ; sa passerelle fait du S-NAT et remplace adresse et port source.

Q50. `e19a` = 57 754 (port source, éphémère) ; `0035` = 53 (destination : DNS) ; `0026` = 38 octets au total, donc 38 − 8 = 30 octets de données ; checksum `0000`. En IPv4, 0 est toujours accepté (pas de contrôle) ; en IPv6 ce n'est plus permis.

Q51. (a) `01bb` = 443, `c738` = 51 000 : le segment vient du **serveur** HTTPS. (b) Sequence `b2d05e01` = 3 000 000 001 ; Acknowledgement `0001e241` = 123 457. (c) Octet 12 `0x80` : Data Offset = 8 mots, en-tête de 32 octets, dont 12 d'options (rappel général). (d) Octet 13 `0x12` = `00010010` : ACK et SYN (rappel général sur l'ordre des bits) : c'est le **SYN, ACK**, 2e segment de la poignée de main. Window Size `ffff` = 65 535. (e) Ack = Sc+1, donc Sc = 123 456. Le client répond ACK avec Sequence = 123 457 et Acknowledgement = Ss+1 = 3 000 000 002.

Q52. (a) SYN (Seq 1000) ; SYN, ACK (Seq 5000, Ack 1001) ; ACK (Seq 1001, Ack 5001). (b) Segments de données : Seq 1001, 1101, 1201 ; acquittements attendus 1101, 1201, 1301. (c) Client FIN ; serveur FIN, ACK ; client ACK (rappel général : souvent quatre segments, FIN et ACK séparés). (d) Rappel général : des numéros prévisibles faciliteraient le détournement de connexion, attaque citée p. 10. SYN flood : beaucoup de SYN sans suite pour remplir la table des connexions à moitié ouvertes du serveur. SYN cookies : le serveur ne retient rien, il calcule Ss par une fonction cryptographique et vérifie l'ACK avec la même fonction.

Q53.

	Proto	Connexion	Unité	Fiabilité	Congestion
TCP	6	oui	flux	oui, ordonné	oui
UDP	17	non	datagramme	non	non
DCCP	33	ouverture fiable	datagramme	ACK sans retransmission	notification seule
SCTP	132	oui	datagramme	oui, ordonné ou non	oui (remplace TCP)

Fiabilité et congestion de SCTP : rappel général (RFC 9260) ; la slide dit « replacement for TCP ». Ouvrir un socket d'un autre protocole que TCP ou UDP demande des privilèges particuliers : au-dessus d'UDP, l'application implémente son propre contrôle sans droits spéciaux.

Q54. (a) Table de traduction de la passerelle (port externe, adresse et port internes) :

Port externe (94.44.112.4)	Interne
40001	192.168.1.31:50000
40002	192.168.1.32:50000
40003	192.168.1.31:50001

(b) 2e connexion :

Tronçon	Source	Destination
interne, aller	192.168.1.32:50000	152.66.244.17:443
externe, aller	94.44.112.4:40002	152.66.244.17:443
externe, retour	152.66.244.17:443	94.44.112.4:40002
interne, retour	152.66.244.17:443	192.168.1.32:50000

(c) Vue du serveur, les deux connexions auraient le même 5-uplet {94.44.112.4, 152.66.244.17, TCP, 50000, 443} : la passerelle ne saurait pas à qui rendre la réponse. (d) Non : 192.168.1.31 n'est pas routable et le S-NAT ne traite que les connexions sortantes ; il faut un D-NAT (ou une ouverture de port par UPnP IGD). (e) Au choix : nombre limité de ports éphémères ; pair à pair et H.323 qui veulent des adresses publiques directes ; signature cryptographique des adresses des extrémités.

10. Corrigés : fenêtres TCP

Source : Q55 à Q57 : `hta5p` p. 7 à 15 ; Q58 : p. 13 et 18 ; Q59 : p. 16, `hta5` p. 16, TD 2019 p. 40 ; Q60 : p. 17, TD 2019 p. 55 ; Q61 : p. 3 à 7. Tous les tableaux viennent de `calc_net_s46.py`

Q55.

Ronde	cwnd	ssthresh	Phase	Fenêtre eff.	Événement
1	1	12	SS	1	
2	2	12	SS	2	
3	4	12	SS	4	
4	8	12	SS	8	
5	12	12	CA	12	
6	13	13	CA	12	
7	14	14	CA	12	

En ronde 5 le doublement donnerait 16, plafonné à `ssthresh` = 12 ; ensuite congestion avoidance. À partir de la ronde 5, la fenêtre effective min(`adv_wnd`, `cwnd`) reste à 12 : c'est le **contrôle de flux** (le récepteur) qui limite, plus la congestion.

Q56.

Ronde	cwnd	ssthresh	Phase	Événement
1	1	8	SS	
2	2	8	SS	
3	4	8	SS	
4	8	8	CA	
5	9	9	CA	
6	10	10	CA	
7	11	11	CA	
8	12	12	CA	
9	13	13	CA	
10	14	14	CA	
11	15	15	CA	
12	16	16	CA	timeout
13	1	8	SS	
14	2	8	SS	
15	4	8	SS	
16	8	8	CA	
17	9	9	CA	
18	10	10	CA	

Au timeout de la ronde 12, `cwnd` = 16 et `ssthresh` = 16 : nouveau `ssthresh` = 8, `cwnd` = 1, slow start. Total : $1+2+4+8+9+\cdots+16+1+2+4+8+9+10 =$ **149** segments.

Q57.

Ronde	cwnd	ssthresh	Phase	Événement
1	1	8	SS	
2	2	8	SS	
3	4	8	SS	
4	8	8	CA	
5	9	9	CA	
6	10	10	CA	
7	11	11	CA	
8	12	12	CA	
9	13	13	CA	
10	14	14	CA	
11	15	15	CA	
12	16	16	CA	3 ACK dupliqués
13	8	8	CA	
14	9	9	CA	
15	10	10	CA	
16	11	11	CA	
17	12	12	CA	
18	13	13	CA	

(b) Total **178** segments, soit 29 de plus ; sur les rondes 13 à 18 : 63 contre 34. (c) La quantité transmise est l'aire sous la courbe de `cwnd` : Reno évite la chute à 1 et la remontée, la courbe reste près du genou (p. 15). (d) Sur timeout, Reno fait comme Tahoe : retour en slow start à `cwnd` = 1 (p. 14).

Q58.

Segment émis	Récepteur	Réponse
20	reçu	ACK 21
21	perdu	
22	reçu	ACK 21 (dupliqué 1)
23	reçu	ACK 21 (dupliqué 2)
24	reçu	ACK 21 (dupliqué 3)
25	reçu	ACK 21 (dupliqué 4)
21	reçu	ACK 26
26	reçu	ACK 27

(a) ACK 21 (pour 20), puis 21 quatre fois (pour 22, 23, 24, 25), puis 26 et 27. (b) Au **3e ACK dupliqué**, déclenché par 24 ; sur la figure la retransmission part après 25, déjà émis. (c) ACK 26 : 22 à 25 étaient déjà en tampon, l'ACK est cumulatif. (d) Des segments peuvent arriver dans le désordre sans être perdus ; attendre 3 doublons évite des retransmissions inutiles. New Reno retransmet dès le premier doublon et garde `cwnd`, au risque de retransmissions excessives en cas de désordre (p. 18).

Q59. Méthode : BDP = débit × RTT ; débit max = W/RTT ; travailler en octets. (a) $100 \times 10^6 \times 0{,}08 = 8 \times 10^6$ bits = 1 000 000 octets, soit $\lceil 10^6/1460 \rceil = 685$ segments. Sans option : $65\,535 \times 8/0{,}08 = 6{,}55$ Mbit/s. $65\,535 \times 2^3 = 524\,280 < 10^6 \leq 65\,535 \times 2^4 = 1\,048\,560$: décalage **4**. (b) $10^{10} \times 0{,}1/8 = 125 \times 10^6$ octets (125 Mo). Sans option : 5,24 Mbit/s, soit 0,05 % du lien. $65\,535 \times 2^{11} = 134\,215\,680 \geq 125 \times 10^6$: décalage **11** ; la limite $65\,535 \times 2^{14} \approx 1{,}07 \times 10^9$ octets suffit. (c) 8 ko : $8192 \times 8/0{,}08 = 0{,}82$ Mbit/s ; 64 ko : 6,55 Mbit/s. La petite fenêtre sature avant le lien, d'où l'intérêt de l'option window scale (TD 2019, p. 40).

Q60. (a) $1500 \times 8 = 12\,000$ bits ; $p \approx 12\,000 \times 10^{-6} = 1{,}2$ %, un paquet sur $1/0{,}012 = 83{,}3$ (valeur du TD ; le calcul exact $1 - (1 - 10^{-6})^{12\,000}$ donne 1,19 %, un sur 83,8). (b) Reno prend toute perte pour de la congestion et divise sa fenêtre, alors que la perte vient du lien, pas d'un goulot. (c) Fenêtre pilotée par le délai, fast retransmit, SACK (New Reno diffère alors la réduction de fenêtre).

Q61. (a) Flux : protège le **récepteur**, champ Window Size (`adv_wnd`, octets libres du tampon). Congestion : protège le **réseau**, variable `cwnd` de l'émetteur. (b) Il s'arrête un moment, puis envoie un petit segment pour obtenir une fenêtre dans l'ACK. (c) Fenêtre annoncée de plus en plus petite (émetteur rapide, ou petites écritures de l'application) ; Nagle côté émetteur, fenêtre non nulle annoncée seulement quand le tampon est à moitié vide côté récepteur. (d) Goodput : trafic utile (arrivé, sans doublon) ; genou : au-delà, goodput non proportionnel à la charge et latence qui explose ; falaise : effondrement. (e) Timeout, ACK dupliqués.

11. Corrigés : Web et e-mail

Source : Q62 : `hta6` p. 8 et 11 ; Q63 : p. 10 et 15 ; Q64 : p. 12 à 14 ; Q65 : `hta6p` p. 4 et 7 à 10 ; Q66 : p. 7 à 10 ; Q67 : p. 5 et 6 ; Q68 : p. 13 à 19

Q62. (a) Succès : 200. Redirection : 301. Erreur client : 403, 404. Erreur serveur : 500, 502, 504. (b) GET récupère l'objet de l'URL ; POST envoie des données à placer à l'URL ; HEAD ne récupère que les métadonnées. (c) Par un cookie : la réponse le pose, le navigateur le renvoie au même site à chaque requête. Cookie tiers : posé pour le compte d'un autre domaine, outil de pistage. Une réponse avec cookie ne peut pas être mise en cache.

Q63. (a) 1.1 : réutilisation de la connexion TCP ; 2 : compression des entêtes et multiplexage des requêtes ; 3 : QUIC sur UDP au lieu de TCP. (b) HTTP/1.0 : $5 \times (1+1) = 10$ RTT $= 500$ ms ; HTTP/1.1 : $1+5 = 6$ RTT $= 300$ ms. (c) TCP + TLS 1.3 : 2 RTT (100 ms) ; QUIC, qui combine transport et TLS (3 échanges au lieu de 6, `hta6` p. 15) : 1 RTT (50 ms).

Q64. (a) 80 et 443. (b) DH établit un secret commun sans le transmettre, mais n'authentifie personne : un intermédiaire peut faire un échange avec chaque côté. (c) Le certificat a été émis pour le site auquel on est connecté, et il est signé par une CA de confiance. (d) Il permet de chiffrer, pas de prouver l'identité du site. (e) Web of trust : confiance transitive entre personnes (PGP) ; PKI : CA reconnues qui signent les clés des sites (X.509). (f) Couche application ; poignée de main TCP d'abord, puis poignée de main TLS.

Q65.

1. MUA d'Alice → MSA de a.example : SMTP en soumission, port 465 (avant 587), TLS, authentification.
2. MTA de a.example : cherche dans le DNS l'enregistrement **MX** de b.example.
3. MTA de a.example → MTA de b.example : SMTP, TCP 25 (relais entre serveurs).
4. Remise au MDA de b.example, où se trouve la boîte de Bob (stockage mbox ou maildir).
5. MUA de Bob → MDA : IMAP, 143 ou 993 avec TLS.

Q66. (a) `MAIL <from>`, `RCPT <to>`, `DATA`. (b) POP3 110 et 995 ; IMAP 143 et 993. POP3 relève (`RETR`) et peut supprimer (`DELE`), boîte de réception seule ; IMAP interroge la base du serveur : dossiers, en-têtes et pièces jointes séparément, copie dans « Sent », relève ou consulte (`FETCH`). (c) Sur un client IMAP.

Q67. (a) Le corps d'un message n'admet que de l'ASCII 7 bits : MIME convertit caractères internationaux et binaire ; quoted-printable, base64. (b) $3 \times 4/3 = 4$ Mo, un peu plus avec les fins de ligne. (c) mbox : tous les messages dans un seul fichier, chaque message commence par `From ` ; maildir : un fichier par message, répertoires `new`, `cur`, `tmp`.

Q68. (a) PLAIN : mot de passe en clair dans une connexion TLS. CRAM : le serveur envoie un aléa, le client renvoie hash(mot de passe, aléa) ; le mot de passe ne circule pas et chaque connexion a un aléa différent : protection contre le rejeu. (b) Un secret partagé entre service et appareil ; code à 6 chiffres valable moins d'une minute ; horloges assez bien synchronisées. (c) DKIM : le MTA émetteur signe le message (clé publique dans le DNS) ; SPF : le MTA prouve qu'il est autorisé à envoyer pour le domaine du From ; DMARC : publie dans le DNS comment DKIM et SPF sont appliqués. (d) Web of trust : on fait confiance à ceux à qui nos personnes de confiance font confiance. (e) Attaques très élaborées, et un utilisateur finit toujours par cliquer sur un lien par erreur.